

EJERCICIO DE EVALUACIÓN TÉCNICA

Sistema de Gestión de Inventario

Desarrollador Junior — Full Stack

Take-home

Contexto

Una pequeña empresa necesita llevar el control de su inventario de productos. El sistema debe permitir registrar productos, gestionar el stock disponible y registrar los movimientos (entradas y salidas) de cada producto.

Antes de empezar, leé esto:

- El objetivo no es que entregues algo perfecto. Nos interesa ver cómo pensás y cómo fundamentás tus decisiones.
- Tampoco buscamos algo sofisticado: queremos ver que comprendés el problema y podés tomar decisiones razonables sobre cómo abordarlo.
- En lo que sea código, lo mínimo que demuestre el manejo de las herramientas y los conceptos involucrados es suficiente.

Las 3 tablas

El sistema se apoya en estas tres entidades. Son el punto de partida para todas las partes del ejercicio.

Tabla	Campos	Descripción
Categoría	id, nombre, descripción	Agrupar los productos (ej: Electrónica, Ropa, Alimentos).
Producto	id, nombre, categoría_id, precio_unitario, stock_actual	Artículo disponible en el inventario.
Movimiento	id, producto_id, tipo (entrada/salida), cantidad, fecha, motivo, usuario	Registro de cada cambio en el stock de un producto.

Reglas de negocio a tener en cuenta:

- El stock de un producto nunca puede quedar en negativo.
- Cada movimiento debe quedar registrado con fecha, tipo y quién lo hizo.
- Un producto pertenece a una sola categoría y siempre debe tener una asignada; no puede quedar sin categoría.
- Todo producto debe tener precio unitario definido; no puede estar vacío o nulo.

1. Base de Datos

1.1 Diseño del modelo

Proponé cómo estructurarías estas tres tablas. Podés hacerlo con un DER, un script SQL, modelos de ORM o como prefieras. Luego respondé:

- ¿Qué tipo de base de datos elegirías y por qué?
- ¿Usarías un ORM? ¿Qué ventajas y desventajas ves en este caso?
- ¿Cómo garantizarías que el stock no quede negativo?
- ¿Cómo conviven estas herramientas dentro del stack tecnológico de un proyecto? ¿Cuáles usarías y por qué? (Por ejemplo: ¿ORM junto con migraciones? ¿Cliente directo en algún caso? ¿Cómo interactúan con el resto de la arquitectura?)

1.2 Historial de cambios

- Si alguien edita el nombre o precio de un producto, ¿cómo guardarías el valor anterior?
- ¿Guardarías esa auditoría en la misma tabla, en una tabla aparte o en otro sistema?

1.3 Para pensar (no implementar)

Si el sistema crece, ¿cómo lo encarás?

- El inventario pasa a tener 500 000 productos con miles de movimientos diarios.
- Se necesita saber el stock histórico de un producto en cualquier fecha pasada.

2. Backend

2.1 Decisiones de arquitectura

- ¿Qué lenguaje y framework usarías? (Preferimos **Python**, pero podés justificar otra elección.)
- ¿Cómo organizarías el proyecto en carpetas? Prestá especial atención a cómo pensás la **orquestación del proyecto**: cómo se conectan las capas entre sí (rutas, servicios, repositorios, modelos), y cómo esa estructura facilita mantener y escalar el sistema.
- ¿Dónde y cómo configurarías las reglas de negocio (ej: stock mínimo, tipos de movimiento válidos)?

2.2 Deploy

- ¿Cómo deployarías este sistema? ¿En la nube, Docker, empaquetado?
- ¿Cómo manejarías las diferencias entre entorno local y producción?

2.3 Seguridad: roles, perfiles y contraseñas

- ¿Cómo manejarías los distintos roles de usuario en el sistema (por ejemplo: administrador, operador de depósito, solo lectura)? ¿Dónde definirías esos permisos?
- ¿Qué consideraciones tendrías para el manejo de contraseñas? (almacenamiento, hashing, políticas mínimas.)
- ¿Cómo protegerías los endpoints según el rol del usuario autenticado?

2.4 Logs del sistema

- ¿Qué tipo de eventos registrarías en los logs? (errores, acciones de usuario, cambios de estado, etc.)
- ¿Dónde guardarías esos logs y con qué herramienta? ¿Archivo local, servicio externo, base de datos?
- ¿Cómo diferenciarías los logs de aplicación de la auditoría de negocio (que ya abordaste en la Parte 1)?

Ahora armo la carpeta SGI_backend

2.5 Endpoint a implementar

Implementá un endpoint que devuelva el listado de productos con su stock actual. Debe:

- Devolver los datos en JSON.
- Permitir filtrar por categoría (ej: `/productos?categoria=electronica`).
- Incluir el nombre de la categoría en la respuesta (no solo el id).

Podés usar datos mockeados si no tenés base conectada. Lo que evaluamos es cómo estructurarás el código.

3. Frontend

3.1 Elección de tecnología

Tenés dos opciones: **React** o **Streamlit**. Respondé:

- ¿Cuál elegirías para este sistema y por qué? *react xq es escalable*
- ¿En qué contexto usarías Streamlit? ¿Y React? *para el caso que no lo utilice muchas personas a la vez y sea para un proyecto pequeño*
- ¿Qué sacrificás con cada uno? *poder de manipulacion y control ?¿*

3.2 Pantallas a diseñar

Vista de tabla de productos:

- ¿Qué columnas mostrás? ¿Por qué esas?
- ¿Cómo indicás visualmente si el stock está bajo?
- ¿Qué acciones tendrían desde esa vista?
- Incorporá un **gráfico de stock por producto** (ej: barras comparando el stock actual de cada producto o por categoría). ¿Qué tipo de gráfico elegirías y por qué?

Alerta de productos con stock bajo:

- Diseña una vista o componente que alerte cuando un producto esté por debajo de cierto umbral de stock.
- ¿Cómo definirías ese umbral? ¿Sería fijo, configurable por categoría, configurable por producto?
- ¿Qué cambios necesitaría el modelo de datos para soportarlo correctamente?
- ¿Qué consideraciones de UX tendrías en cuenta para no generar ruido con alertas que el usuario termina ignorando?

Formulario para registrar un movimiento (entrada o salida):

- ¿Qué campos incluís?
- ¿Cómo validás que no quede stock negativo antes de enviar?
- ¿Qué botones ponés y dónde? Defendé esa decisión.

Podés entregar mockup digital o código. Lo que importa es que expliques el criterio detrás de cada decisión.

4. Git

- Todo el trabajo debe estar en un repositorio (GitHub).
- Se recomienda utilizar buenas prácticas de Git. Puede seguir un flujo tipo Gitflow (ramas por feature, develop, main) o el esquema que prefiera, siempre que el historial de commits sea claro y el criterio de ramificación tenga sentido.

No evaluamos la cantidad de commits sino la claridad. Un buen mensaje de commit vale más que diez que digan “fix”.

5. Uso de IA

Es válido y esperado que uses IA (ChatGPT, Claude, Copilot, etc.). Solo contanos:

- ¿Qué herramienta usaste?
- ¿Para qué la usaste puntualmente?
- ¿Hubo algo que te sugirió y decidiste no usar o cambiar? ¿Por qué?

No hay respuesta correcta aquí. Queremos entender cómo integrás estas herramientas a tu trabajo.

Entrega

Compartí el link al repositorio con acceso habilitado. El repo debe incluir:

- Código del endpoint de backend.
- Diseño o código del frontend (puede ser parcial).
- Modelo de datos (SQL, ORM, DER o lo que hayas elegido).
- README con instrucciones y decisiones tomadas.
- Respuestas a las preguntas conceptuales (en el README o en un doc aparte).

Ante cualquier duda sobre el enunciado, escribinos.
Preferimos que preguntes a que supongas algo incorrecto.